

POLITECNICO

MILANO 1863

PROVA FINALE

PROGETTO DI RETI LOGICHE

Prof. William Fornaciari

CORSO DI LAUREA IN INGEGNERIA INFORMATICA

Anno accademico: 2023/2024

Alessandro Del Fatti

Codice persona: 10790553

Consegna del 01/04/2024

Indice

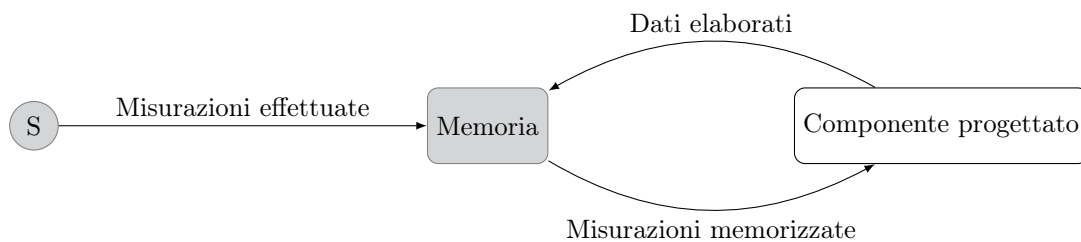
1	Introduzione	2
2	Architettura	3
2.1	Implementazione in VHDL	3
2.2	Macchina a stati finiti	5
2.2.1	INIT	5
2.2.2	READY	6
2.2.3	MEM_READ_FIRST	6
2.2.4	W_MEM_PREP_FIRST	6
2.2.5	UPDATE_W	6
2.2.6	C_MEM_PREP	6
2.2.7	C_UPDATE	6
2.2.8	MEM_READ	7
2.2.9	W_MEM_PREP	7
2.2.10	DONE	7
3	Risultati sperimentali	7
3.1	Test basati sugli esempi delle specifiche	7
3.1.1	Esempi 1, 4, 5	7
3.1.2	Esempio 2	7
3.1.3	Esempio 3	7
3.2	Test su casi limite - valori	8
3.2.1	Test 1	8
3.2.2	Test 2	8
3.2.3	Test 3	8
3.2.4	Test 4	8
3.2.5	Test 5	8
3.3	Test su casi limite - comportamenti	8
3.3.1	Test 1	8
3.3.2	Test 2	9
3.3.3	Test 3	9
3.3.4	Test 4	9
4	Conclusioni	9

1 Introduzione

Il sistema progettato ha come scopo quello di ricevere in ingresso dei valori interi positivi, valutare la loro attendibilità e generare, in modo coerente con il valore letto, un ulteriore valore numerico che possa descriverne la credibilità. Si pensi, ad esempio, ad un sistema che opera in questo modo:

1. Un sensore effettua una serie di misurazioni: sia K il numero di misurazioni effettuate.
2. Le K misurazioni vengono mantenute in delle celle di memoria. Sia W il contenuto di ciascuna cella di memoria rappresentante una misurazione.
3. Per ciascuna delle K misurazioni si vuole dire se il valore memorizzato è attendibile oppure no. Si considerano attendibili valori di $W > 0$ e non attendibili valori di $W = 0$.
4. Quindi, per ciascuna misurazione, si assegna un valore di credibilità C secondo queste regole:
 - (a) Se $W_i \neq 0$, il corrispondente valore di credibilità C_i sarà massimo, cioè $C_i = 31$
 - (b) Se $W_i = 0$, la misurazione è da considerarsi non valida. Si sostituisce quindi il valore di W_i con quello di W_{i-1} e si pone C_i a $C_{i-1} - 1$, se $C_{i-1} \geq 1$ altrimenti si pone $C_i = 0$. Se già il primo valore letto è nullo, cioè $W_0 = 0$, non si può «correggere» con la lettura precedente e si mantengono $W_0 = 0$ e $C_0 = 0$.

Il seguente schema esplicita in modo generale il principio di funzionamento del sistema e dove si colloca il componente progettato.

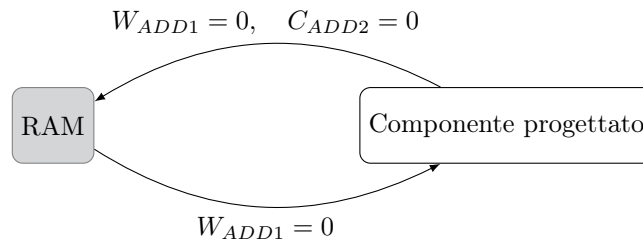


Un esempio di elaborazione, con valori numerici, che racchiude le casistiche di input descritte è il seguente:

Indirizzo	Valore pre-elaborazione	Valore post-elaborazione
ADD_1	0	0
ADD_2	0	0
ADD_3	192	192
ADD_4	0	31
ADD_5	0	192
ADD_6	0	30

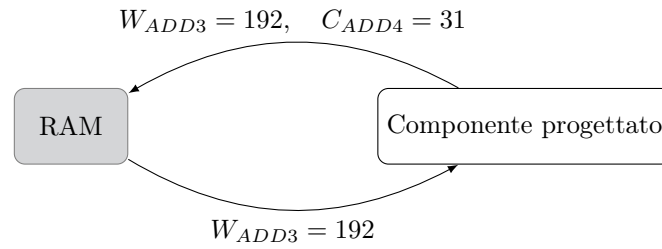
Tabella 1: Situazione della memoria pre e post-elaborazione

Passo 1: Elaborazione del primo valore



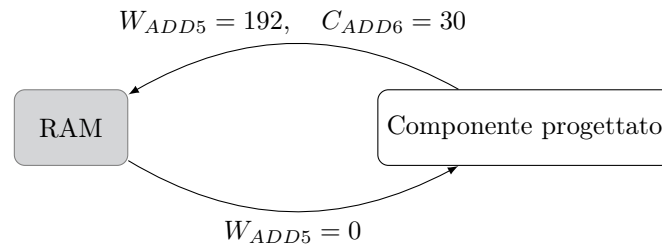
Leggo 0 dalla memoria. Si tratta del primo valore letto, quindi non posso «correggerlo» con quello letto al passo precedente.

Passo 2: Elaborazione del secondo valore



Leggo 192 dalla memoria. Si tratta di un valore maggiore di 0, quindi lo considero attendibile e imposto al massimo il valore di credibilità $C = 31$.

Passo 3: Elaborazione del terzo valore



Leggo 0 dalla memoria. Non si tratta del primo valore letto, quindi non lo considero attendibile e riscrivo in memoria il valore di W letto al punto precedente, ma abbasso il livello di credibilità: $C = 30$.

I valori numerici di questo esempio sono coerenti con le specifiche: K , cioè il numero di letture che il componente deve effettuare, è esprimibile con 10 bit, W , il dato in memoria, è una parola da 8 bit e gli indirizzi di memoria sono da 16 bit.

2 Architettura

Il componente progettato, mostrato in Figura 1 è stato suddiviso in dei sottoblocchi funzionali, ognuno con un compito diverso:

- **ADDR-REG**, è un registro mantiene indica l'indirizzo della cella di memoria sulla quale effettuare le operazioni di lettura e scrittura;
- **K-VAL-REG**, è un registro che viene inizializzato con il valore K fornito in ingresso. Indica cioè il numero di operazioni di lettura e scrittura ancora da effettuare;
- **W-REG**, è un insieme di due registri che mantengono memorizzate, rispettivamente, la parola W e la credibilità C ;
- **FSM**, è la macchina a stati che funziona da unità di controllo. Abilita e disabilita le uscite (*e di conseguenza i rispettivi ingressi degli altri componenti*) in funzione dello stato in cui si trova. La descrizione degli stati e dei segnali di uscita è riportata

Nello schema è presente anche un **multiplexer**, necessario per portare in uscita il valore di W o di C .

2.1 Implementazione in VHDL

La struttura appena descritta, e riportata in Figura 1 è stata anche mappata nel codice VHDL, di cui si riportano le entity:

```

1 entity addr_reg is
2   port (
3     i_addr      : in std_logic_vector(15 downto 0);
4     i_clk       : in std_logic;
5     i_rst       : in std_logic;
6     en_add_inc  : in std_logic;
  
```

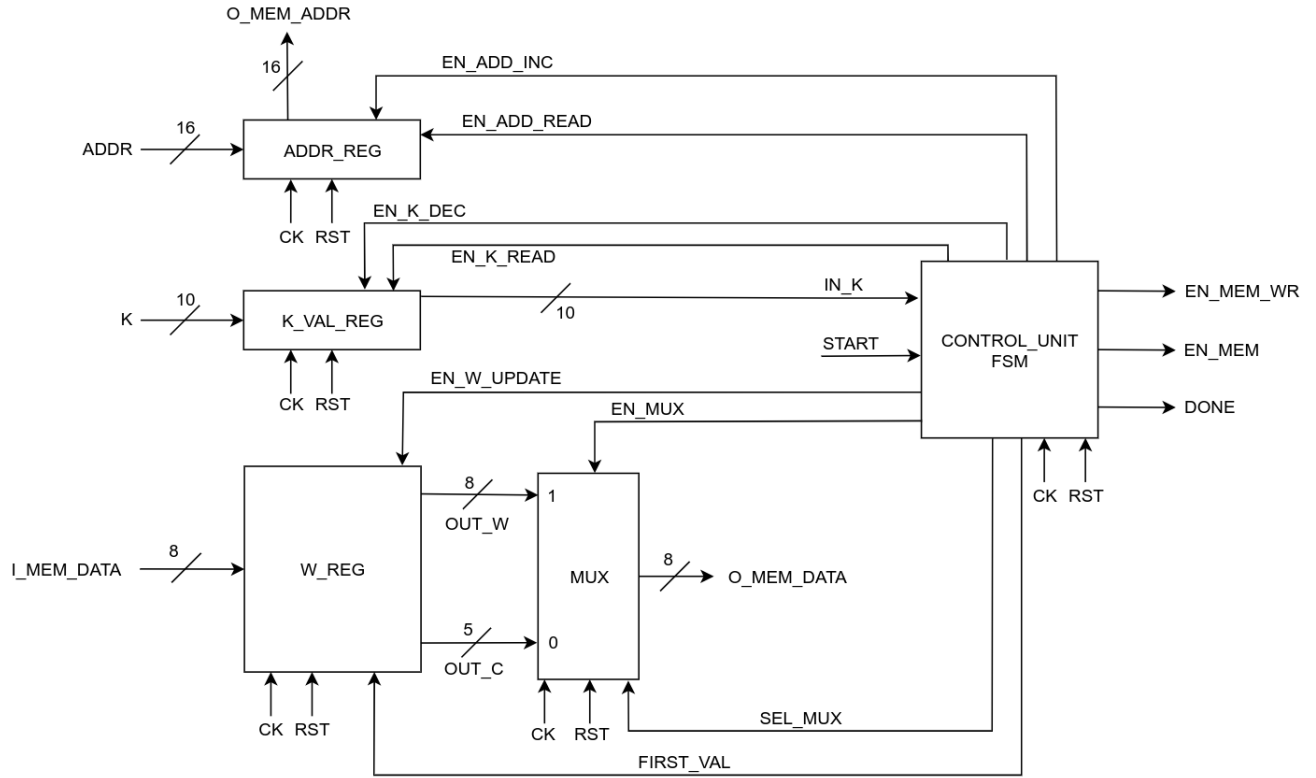


Figura 1: Schema a blocchi del componente progettato

```

7   en_add_read : in std_logic;
8   o_mem_addr  : out std_logic_vector(15 downto 0)
9 );
10 end entity addr_reg;

```

In cui, `i_addr` è un vettore di bit inizializzato con il primo indirizzo di memoria dal quale iniziare l'elaborazione e `o_mem_addr` è l'indirizzo corrente e indica qual è la cella di memoria da leggere o scrivere ; viene di volta in volta incrementato (avendo definito un opportuno segnale nell'architettura, dato che essendo `o_mem_addr` un output non può essere scritto).

```

1 entity k_val_reg is
2   port(
3     i_k      : in std_logic_vector(9 downto 0);
4     i_clk    : in std_logic;
5     i_rst    : in std_logic;
6     en_k_read : in std_logic;
7     en_k_dec  : in std_logic;
8     o_k_val   : out std_logic_vector(9 downto 0)
9   );
10 end entity k_val_reg

```

In cui, `i_k` è un vettore di bit inizializzato con la lunghezza K della sequenza da elaborare. `o_k_val` è invece il valore corrente di K , che viene decrementato al termine di ogni elaborazione fino a diventare zero. Anche in questo caso, nell'architettura è stato introdotto un apposito segnale che permette la riscrittura di K .

```

1 entity w_reg is
2   port(
3     i_mem_data : in std_logic_vector(7 downto 0);
4     i_clk      : in std_logic;
5     i_rst      : in std_logic;
6     i_first_val : in std_logic;
7     en_w_update : in std_logic;
8     out_w       : out std_logic_vector(7 downto 0);
9     out_c       : out std_logic_vector(4 downto 0)
10  );
11 end entity w_reg;

```

In cui, `i_mem_data` è il vettore di 8 bit con il dato proveniente dalla memoria e che deve essere elaborato secondo i requisiti di funzionamento specificati nell'introduzione. `out_w` e `out_c` sono i due vettori post-elaborazione, rispettivamente da 8 e 5 bit, e che devono essere scritti in memoria. Anche in questo caso, nell'architettura è stato introdotto un apposito segnale che permette la riscrittura di tali output.

```

1 entity mux is
2   port(
3     in_w      : in std_logic_vector(7 downto 0);
4     in_c      : in std_logic_vector(4 downto 0);
5     en_mux    : in std_logic;
6     sel_mux   : in std_logic;
7     i_clk     : in std_logic;
8     i_rst     : in std_logic;
9     o_mem_data : out std_logic_vector(7 downto 0)
10  );
11 end entity mux;

```

In cui, `in_w` e `in_c` sono i segnali provenienti dal blocco W-REG e `o_mem_data` è il vettore da 8 bit che deve essere scritto in memoria assumendo, alternativamente, un valore di *W* o di *C*. Nel secondo caso, il multiplexer si occupa anche di fare un'operazione di padding per portare `in_c`, vettore da 5 bit, su `o_mem_data`, vettore da 8 bit.

Tutte le architetture di questi quattro componenti sono formate da un unico process che ha i soli segnali di clock e di reset nella sensitivity list. L'architettura della macchina a stati, descritta nel seguito, è invece formata da due process differenti:

- Un process che descrive la **funzione di stato prossimo**, sequenziale, e ha nella sensitivity list i segnali di clock e di reset;
- Un process che descrive la **funzione di uscita**, combinatorio, e ha nella sensitivity list il segnale di stato corrente.

2.2 Macchina a stati finiti

Omettendo di riportare il codice VHDL della entity dell'FSM, di seguito è invece schematizzato il comportamento di transizione tra i diversi stati della macchina:

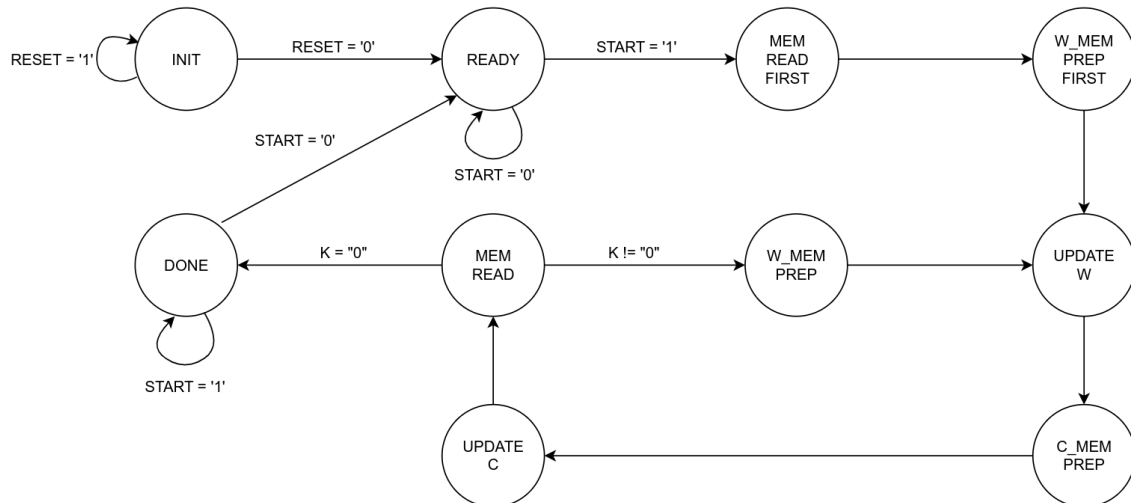


Figura 2: Macchina a stati finiti

Essendo una macchina di Moore, a ciascuno stato sono anche specificate delle uscite. Per una migliore leggibilità, il valore delle uscite è riportato in Tabella 2. Di seguito sono invece descritti i vari stati:

2.2.1 INIT

Lo stato di INIT è quello che il componente raggiunge non appena viene dato il segnale di reset. Sarà quindi lo stato iniziale della prima elaborazione.

	o_done	o_en_mux	o_sel_mux	o_w_update	o_k_read	o_k_dec	o_add_read	o_add_inc	o_en_mem	o_mem_wr	o_first_val
INIT	0	0	0	0	0	0	0	0	0	0	0
READY	0	0	0	0	1	0	1	0	0	0	0
MEM_READ_FIRST	0	0	0	0	0	0	0	0	1	0	0
W_MEM_PREP_FIRST	0	0	0	1	0	0	0	0	0	0	1
MEM_READ	0	0	0	0	0	0	0	0	1	0	0
W_MEM_PREP	0	0	0	1	0	0	0	0	0	0	0
UPDATE_W	0	1	1	0	0	0	0	0	0	0	0
C_MEM_PREP	0	1	0	0	0	0	0	1	1	1	0
UPDATE_C	0	0	0	0	0	1	0	1	1	1	0
DONE	1	0	0	0	0	0	0	0	0	0	0

Tabella 2: Uscite per ciascuno stato della FSM

2.2.2 READY

Lo stato di READY si raggiunge non appena il segnale di reset viene portato a zero. Rappresenta lo stato in cui la macchina è in attesa del segnale alto di start, per poter iniziare l'elaborazione. Mentre il componente è in questo stato, inoltre, inizia la lettura dei valori di K e dell'indirizzo iniziale di memoria.

2.2.3 MEM_READ_FIRST

Lo stato di MEM_READ_FIRST si raggiunge non appena viene dato il segnale di start. In questo stato viene abilitata la lettura dalla memoria, così che al colpo di clock successivo il componente possa lavorare sul valore letto.

2.2.4 W_MEM_PREP_FIRST

Lo stato di W_MEM_PREP_FIRST abilita l'uscita o_w_update per consentire al componente di valutare il valore di W appena letto dalla memoria. Differisce dallo stato di W_MEM_PREP solo per il fatto che il tipo di elaborazione che deve essere svolta, trattandosi del primo valore letto, è diversa da quella da svolgere per le letture successive alla prima.

2.2.5 UPDATE_W

Lo stato di UPDATE_W abilita il processo di aggiornamento del valore di W che dovrà essere scritto in memoria. Le uscite permettono quindi di portare su o_mem_data, (abilitando il multiplexer), al colpo di clock successivo, il nuovo valore di W da scrivere in memoria.

2.2.6 C_MEM_PREP

Lo stato di C_MEM_PREP abilita le uscite per poter scrivere in memoria il valore appena portato su o_mem_data. Contestualmente, attiva anche il segnale per abilitare l'incremento dell'indirizzo (dato che la credibilità, al passo successivo, andrà scritta nell'indirizzo successivo) e per portare su o_mem_data il nuovo valore di credibilità memorizzato in W-REG.

2.2.7 C_UPDATE

Lo stato di C_UPDATE abilita le uscite per poter scrivere in memoria il valore della credibilità appena portato su o_mem_data. Contestualmente, abilita anche l'incremento dell'indirizzo per la successiva scrittura ed abilita il decremento di K , dato che questa elaborazione si è conclusa.

2.2.8 MEM_READ

Lo stato di MEM_READ valuta il valore di K per capire se bisogna continuare con l'elaborazione. Contestualmente, abilita l'uscita per la lettura della memoria (preparandosi quindi a ricevere il nuovo valore da elaborare).

2.2.9 W_MEM_PREP

Lo stato di W_MEM_PREP si raggiunge se $K > 0$. Abilita l'uscita `o_w_update` per consentire al componente di valutare il valore di W .

2.2.10 DONE

Lo stato di DONE si raggiunge quando $K = 0$, cioè quando l'elaborazione è terminata. Si rimane in questo stato fino a che `start` rimane alto. Non appena `start` viene portato a 0, la macchina torna di nuovo in uno stato di READY.

3 Risultati sperimentali

Per testare il componente progettato, come primo passo, è stato sottoposto al testbench fornito insieme alla specifica. Completata correttamente la sintesi e terminate le due simulazioni, behavioral e post-synthesis sono stati generati i `report_timing` e `report_utilization` con i seguenti risultati:

- **Slack (MET):** 15.847ns (`required time` - `arrival time`). Sono quindi stati rispettati i vincoli temporali imposti dalla specifica.
- **Register as Flip Flop:** 51, **Register as Latch:** 0. Da cui si nota che sono stati utilizzati 51 flip-flop e nessun latch.

Terminata senza errori la simulazione del testbench fornito, una secondo passo è stato quello di ricostruire i testbench sulla base degli esempi dello stato della memoria presenti nel file delle specifiche. Come terzo ed ultimo passo nella fase di test sono stati individuati dei casi limite legati sia ai valori assumibili dai vari segnali, sia al comportamento del componente.

Di seguito, per ciascun test effettuato, sono descritte le peculiarità che hanno portato alla scelta di sottoporre il componente a tale test.

3.1 Test basati sugli esempi delle specifiche

3.1.1 Esempi 1, 4, 5

- $K = 10$ e $K = 16$ elaborazioni da eseguire
- Primo valore non nullo
- Presenza di valori nulli interni alla sequenza

3.1.2 Esempio 2

- $K = 35$ elaborazioni da eseguire
- Primo valore non nullo
- Presenza di valori nulli interni alla sequenza. In particolare sono presenti più di 31 valori nulli. Il test va quindi a verificare che, raggiunto un valore di credibilità pari a 0, il componente continua a comportarsi correttamente lasciando nullo il valore di credibilità.

3.1.3 Esempio 3

- $K = 33$ elaborazioni da eseguire
- Primo valore nullo. Il test va quindi a verificare che, letto il primo 0, il valore che viene scritto in memoria sia 0 sia per la parola W , sia per la credibilità.
- Presenza di valori nulli interni alla sequenza

3.2 Test su casi limite - valori

3.2.1 Test 1

- $K = 10$ elaborazioni da eseguire
- Primo valore non nullo
- Non ci sono valori nulli interni alla sequenza

L'obiettivo di questo test è quello di verificare che il comportamento del componente si mantenga corretto qualora i valori di W siano tutti massimi, ovvero in memoria ci siano solo valori pari a 255.

3.2.2 Test 2

- $K = 32$ elaborazioni da eseguire
- Primo valore nullo
- Presenza di valori nulli interni alla sequenza

L'obiettivo di questo test è quello di verificare che il comportamento del componente si mantenga corretto qualora i valori di W siano tutti minimi, ovvero in memoria ci siano solo valori pari a 0.

3.2.3 Test 3

- $K = 1$ elaborazioni da eseguire
- Valore non nullo

L'obiettivo di questo test è quello di verificare che il comportamento del componente si mantenga corretto qualora il numero di elaborazioni da eseguire sia minimo, ovvero $K = 1$.

3.2.4 Test 4

- $K = 1023$ elaborazioni da eseguire
- Valori della sequenza generati in modo pseudo-casuale

L'obiettivo di questo test è quello di verificare che il comportamento del componente si mantenga corretto qualora il numero di elaborazioni da eseguire sia massimo, ovvero $K = 1023$.

3.2.5 Test 5

- $K = 10$
- Indirizzo di partenza $ADD = 0$
- Valori della sequenza generati in modo pseudo-casuale

L'obiettivo di questo test è quello di verificare che il comportamento del componente si mantenga corretto qualora l'indirizzo di partenza da cui partire con l'elaborazione sia 0.

3.3 Test su casi limite - comportamenti

3.3.1 Test 1

- Più segnali di start forniti
- Indirizzi di partenza uguali per due sequenze
- Valori delle sequenze generati in modo pseudo-casuale

Obiettivo di questo test è quello di permettere al componente di effettuare più elaborazioni senza passare dallo stato di reset, se non per la prima sequenza. Il componente deve quindi completare le elaborazioni di tutte le sequenze.

3.3.2 Test 2

- Più segnali di start forniti
- Segnale di reset fornito durante una elaborazione
- Valori delle sequenze generati in modo pseudo-casuale

Obiettivo di questo test è quello di permettere al componente di effettuare più elaborazioni e, durante una elaborazione viene fornito il segnale di reset.

3.3.3 Test 3

- Segnale di start e di reset contemporaneamente alti
- Valori delle sequenze generati in modo pseudo-casuale

Obiettivo di questo test è quello di verificare che, mentre il segnale di reset è alto, nel caso in cui venga fornito il segnale di start, il componente non inizi l'elaborazione ma rimanga in uno stato di reset.

3.3.4 Test 4

- Più segnali di reset forniti
- Valori delle sequenze generati in modo pseudo-casuale

Obiettivo di questo test è quello di verificare che il componente si comporti in modo corretto qualora vengano forniti segnali di reset in diversi stati dell'elaborazione (durante l'elaborazione, come per il test 2 e post-elaborazione).

4 Conclusioni

Il componente progettato soddisfa i requisiti temporali imposti dalle specifiche e simula correttamente i casi di test sopra elencati, sia in behavioral che in post-sintesi.

La scelta di una divisione in più moduli del componente ha semplificato la progettazione, e la conseguente scrittura del codice VHDL, oltre alla gestione dei vari segnali per garantire il funzionamento nei diversi stati di elaborazione.